INFORME DE GRUPO

- **Nombre:** Juliana Valentina Dulce Rios - Maria Valentina Arias Cubillos
- **Fecha:** 22 de octubre de 2026
- **Tema:** Taller con Git y GitHub: Ramas, conflictos y rescate de código
- **Link repositorio:** https://github.com/igibfy11/ActividadClase22.git
- **Link video explicativo:** https://drive.google.com/file/d/1Ir5oGohUZx9-YZraR0IEtT1-rW4KfzQZ/view?usp=sharing

1. Introducción

El control de versiones es una herramienta clave cuando trabajamos en equipo, ya que nos ayuda a organizar el código y evitar enredos. En este taller pusimos en práctica cómo usar ramas en Git y, lo más importante, cómo reaccionar y rescatar el proyecto cuando la rama principal (master) se daña por un conflicto al momento de unir el trabajo de todos.

2. Objetivo

- Aprender a usar un flujo de trabajo ordenado en Git y GitHub utilizando ramas individuales para cada funcionalidad.
- Entender cómo solucionar problemas y errores cuando se integra el código en el repositorio central.
- Investigar y aplicar comandos de emergencia clave para recuperar una versión estable del proyecto.

3. Desarrollo

El taller lo realizamos paso a paso siguiendo estas etapas:

1. **Creación de la base:** La líder del grupo creó el repositorio en GitHub, organizó la carpeta llamada Taller-emergencia y subió el archivo principal de la calculadora a la rama master con el mensaje de confirmación (*commit*): *"versión de la calculadora estable"*.

2. **Trabajo en ramas individuales:** Cada una creó su propia rama para desarrollar la parte que le correspondía (la lógica de las operaciones y el diseño) sin afectar el trabajo de la otra. Después, subimos los cambios al repositorio remoto.
3. **El conflicto en master:** Para cumplir con el ejercicio del taller, al momento de hacer la fusión (*merge*), la segunda persona en integrar el código generó un fallo que dañó la rama master, simulando una situación de emergencia real.
4. **Uso de los comandos de rescate:** Para solucionar el problema, investigamos y utilizamos dos comandos fundamentales:
    - **git log --oneline:** Este comando nos sirvió para ver el historial de los *commits* de forma muy resumida en una sola línea. Con esto pudimos encontrar rápido el código (*hash*) del momento exacto en que la calculadora funcionaba perfectamente.
    - **git reset --hard:** Funciona como un botón para regresar en el tiempo. Al aplicarlo junto con el código del *commit* estable, eliminamos el error que había en master y dejamos el archivo limpio y funcionando tal como estaba antes.
5. **Solución final:** La persona encargada aplicó la corrección en el repositorio y dejó la rama principal lista y estable.

4. Conclusiones Unificadas

- Trabajar con ramas separadas nos enseñó a ser más organizadas y a evitar muchos problemas al momento de unir nuestro código en equipo.
- Conocer y saber usar comandos de emergencia como git log --oneline y git reset --hard nos da mucha tranquilidad, ya que aprendimos que cualquier error en Git tiene solución si aplicamos las herramientas adecuadas.